

CSD622 · LECTURE 2

Pattern Recognition & Data Mining

Deep Dive — Continuing from Lecture 1

Covers: Naive Bayes · Decision Trees · SVM · Training NNs · RNNs/LSTMs · Spectral Clustering · FP-Growth Mining · Text Classification · Evaluation Deep Dive

Tutor: Dr A.J Fofanah



What We Cover in Lecture 2



2.1 Naive Bayes — In Depth

Worked text classification example, Laplace smoothing, log-probability trick, Gaussian NB derivation.



2.2 Decision Trees — Advanced

Information gain worked example, CART vs ID3, pruning strategies, regression trees.



2.3 SVM — Mathematical Derivation

Lagrangian dual, KKT conditions, support vectors, C & γ hyperparameter effects.



2.4 Training Neural Networks

Vanishing gradient, optimisers (SGD, Adam, RMSProp), regularisation, batch normalisation.



2.5 RNNs, LSTMs & Transformers

LSTM gate equations, sequence-to-sequence models, attention mechanism, BERT overview.



2.6 Advanced Clustering & Evaluation

Spectral clustering, GMMs, EM algorithm, ROC/PR curves, statistical significance testing.



LECTURE 2 · SECTION 2.1

Naive Bayes Classifier — Deep Dive

Worked example · Laplace smoothing · Gaussian NB · log-probabilities

Spam Classifier — Step by Step

Training Corpus (5 documents):

ID	Text	Label
T1	free money now	Spam
T2	win a free prize	Spam
T3	meeting at noon	Ham
T4	project deadline tomorrow	Ham
T5	free lunch at noon	Ham

Step 1 — Priors

Total docs = 5

Spam = 2 $\rightarrow P(\text{Spam}) = 2/5 = 0.40$

Ham = 3 $\rightarrow P(\text{Ham}) = 3/5 = 0.60$

Step 2 — Word Likelihoods $P(\text{word} \mid \text{class})$ with Laplace Smoothing ($\alpha = 1$)

Spam vocabulary: {free:2, money:1, now:1, win:1, prize:1} — Total spam words = 6, Vocabulary size $|V| = 9$

$P(\text{"free"} \mid \text{Spam}) = (2+1)/(6+9) = 3/15 = 0.20$ $P(\text{"free"} \mid \text{Ham}) = (1+1)/(7+9) = 2/16 = 0.125$

Laplace smoothing: $P(w_j \mid c) = (\text{count}(w_j, c) + \alpha) / (\sum \text{counts in class } c + \alpha \cdot |V|)$. Prevents zero probabilities for unseen words.

Classifying a New Email: "free prize now"

SPAM

$$\log P(\text{Spam}) = \log(0.40) = -0.916$$

$$\log P(\text{"free"} | \text{Spam}) = \log(3/15) = -1.609$$

$$\log P(\text{"prize"} | \text{Spam}) = \log(2/15) = -2.015$$

$$\log P(\text{"now"} | \text{Spam}) = \log(2/15) = -2.015$$

$$\text{log score}(\text{Spam}) = -6.555$$

HAM

$$\log P(\text{Ham}) = \log(0.60) = -0.511$$

$$\log P(\text{"free"} | \text{Ham}) = \log(2/16) = -2.079$$

$$\log P(\text{"prize"} | \text{Ham}) = \log(1/16) = -2.773$$

$$\log P(\text{"now"} | \text{Ham}) = \log(1/16) = -2.773$$

$$\text{log score}(\text{Ham}) = -8.136$$

Decision: $-6.555 > -8.136 \rightarrow$ Spam wins $\rightarrow$ Email classified as **SPAM** ✓

Using log-probabilities avoids numerical underflow when multiplying many small probabilities together.

Gaussian Naive Bayes for Continuous Features

$$P(x_j | c) = (1/\sqrt{2\pi\sigma^2_{jc}}) \cdot \exp(-(x_j - \mu_{jc})^2 / (2\sigma^2_{jc}))$$

Training

For each class c and feature j :

- μ_{jc} = mean of feature j values for class c
- σ^2_{jc} = variance of feature j values for class c

Prediction

For each class c :

- Compute $\log P(c) + \sum_j \log P(x_j | c)$
- Use Gaussian formula for $P(x_j | c)$ above

Example

Iris dataset: 4 features (sepal/petal length/width)

For class 'Setosa': $\mu = [5.0, 3.4, 1.5, 0.2]$, σ^2 estimated per feature

New flower $x = [5.1, 3.5, 1.4, 0.2] \rightarrow$ score each class

Variants summary: Gaussian NB $\rightarrow$ continuous features | Multinomial NB $\rightarrow$ word counts | Bernoulli NB $\rightarrow$ binary word presence | Complement NB $\rightarrow$ imbalanced text



LECTURE 2 · SECTION 2.2

Decision Trees — Advanced Topics

*Information gain worked example · CART vs ID3 · Pruning ·
Regression trees*

Worked Example: Choosing the Best Split

Dataset: 14 samples, target = Play Tennis (9 Yes / 5 No)

$$H(S) = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = -0.643 \times (-0.637) - 0.357 \times (-1.485) = 0.940 \text{ bits}$$

Feature: Outlook (Sunny / Overcast / Rainy)

Sunny: 2 Yes, 3 No (5 samples) → $H = 0.971$ bits

Overcast: 4 Yes, 0 No (4 samples) → $H = 0.000$ bits (pure!)

Rainy: 3 Yes, 2 No (5 samples) → $H = 0.971$ bits

$$IG(S, \text{Outlook}) = 0.940 - [(5/14) \times 0.971 + (4/14) \times 0.000 + (5/14) \times 0.971] = 0.940 - 0.693 = 0.247 \text{ bits}$$

$IG(\text{Humidity}) = 0.151 \text{ bits}$ | $IG(\text{Wind}) = 0.048 \text{ bits}$ → Outlook wins! Best first split.

CART vs ID3 vs C4.5 — Key Differences

Aspect	ID3	C4.5	CART
Splitting criterion	Information Gain	Gain Ratio (normalised IG)	Gini Impurity
Feature types	Categorical only	Categorical + continuous	Both (+ regression)
Tree structure	Multi-way splits	Multi-way splits	Binary splits only
Pruning	No pruning	Post-pruning (CF pruning)	Cost-complexity (α tuning)
Missing values	Not handled	Uses fractional samples	Surrogate splits
Output	Classification only	Classification only	Classification + Regression
Used by scikit-learn	No	No	Yes (default)

Gain Ratio (C4.5): $\text{GainRatio}(S, A) = \text{IG}(S, A) / \text{SplitInfo}(S, A)$ where $\text{SplitInfo} = -\sum_v (|S_v|/|S|) \log_2(|S_v|/|S|)$.
Penalises features with many values (e.g. unique IDs).

Rule: Use CART (sklearn) for practice; understand ID3/C4.5 for exams. All are equivalent in principle — criteria differ in bias towards number of splits.



LECTURE 2 · SECTION 2.3

SVM — Mathematical Derivation & Practice

Lagrangian dual · Support vectors · C and γ hyperparameters · Multi-class SVMs

From Primal to Dual — Why Support Vectors?

Primal Problem

Minimise: $(1/2) ||w||^2$
Subject to: $y_i(w \cdot x_i + b) \geq 1 \quad \forall i$

N constraints — one per training sample

Lagrangian

$L(w, b, \alpha) = (1/2) ||w||^2 - \sum_i \alpha_i [y_i(w \cdot x_i + b) - 1]$
where $\alpha_i \geq 0$ are Lagrange multipliers

KKT conditions: $\alpha_i [y_i(w \cdot x_i + b) - 1] = 0$

Dual Problem

Maximise: $\sum_i \alpha_i - (1/2) \sum_i \sum_j \alpha_i \alpha_j y_i y_j x_i \cdot x_j$
Subject to: $\sum_i \alpha_i y_i = 0, \quad \alpha_i \geq 0$

Support Vectors — The Key Insight

From KKT: $\alpha_i > 0$ only when $y_i(w \cdot x_i + b) = 1$ — i.e., points ON the margin boundary. These are the support vectors.
 $w = \sum_i \alpha_i y_i x_i$ — the weight vector is a linear combination of ONLY the support vectors. All other training points are irrelevant.

Tuning C (Soft Margin) and γ (RBF Kernel)

Hyperparameter C — Controls the Bias-Variance Trade-off

- | | |
|---|--|
| C $\rightarrow$ 0 (tiny): | Very wide margin · many misclassifications allowed · high bias, low variance · underfits |
| C = 1 (default): | Balanced trade-off · good starting point for most problems · moderate margin |
| C $\rightarrow$ ∞ (huge): | Hard margin · zero tolerance for misclassification · narrow margin · overfits on noise |

Hyperparameter γ (gamma) — Controls the RBF Kernel's Reach

$K(x,z) = \exp(-\gamma \|x-z\|^2)$ $\gamma = 1/(2\sigma^2)$ — inverse of the Gaussian width

- Small $\gamma \rightarrow$ wide Gaussian $\rightarrow$ each training point influences a large region $\rightarrow$ smoother decision boundary $\rightarrow$ may underfit
- Large $\gamma \rightarrow$ narrow Gaussian $\rightarrow$ each training point influences only its immediate neighbourhood $\rightarrow$ wiggly boundary $\rightarrow$ may overfit

Best practice: use GridSearchCV over $C \in [0.01, 0.1, 1, 10, 100]$ and $\gamma \in [0.001, 0.01, 0.1, 1]$ with stratified 5-fold CV.

Multi-Class SVM & scikit-learn Code

One-vs-Rest (OvR)

Train K classifiers, one per class c vs all others.
Prediction: class with highest decision function score.
Faster but class imbalance per sub-problem.

One-vs-One (OvO)

Train $K(K-1)/2$ classifiers for every pair of classes.
Prediction: majority vote across all pair classifiers.
More classifiers but each is balanced. sklearn default.

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

SVM MUST have features scaled — always use a Pipeline

```
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', probability=True))
])
```

Grid search over C and γ

```
param_grid = {'svm__C': [0.1, 1, 10, 100],
              'svm__gamma': ['scale', 0.01, 0.1]}
gs = GridSearchCV(pipe, param_grid, cv=StratifiedKFold(5),
                  scoring='f1_weighted', n_jobs=-1)
gs.fit(X_train, y_train)
print(gs.best_params_, gs.best_score_)
```



LECTURE 2 · SECTION 2.4

Training Neural Networks — Deep Dive

Vanishing gradient · Optimisers · Regularisation · Batch Norm · Practical tips

Vanishing & Exploding Gradients

$$\partial L / \partial w^l = \partial L / \partial a^L \cdot \prod_{k=l+1}^L (f'(z^k) \cdot W^k) \quad \leftarrow \text{product of } L-l \text{ terms}$$

Vanishing Gradient

Cause:

Sigmoid/Tanh derivatives $\in (0, 0.25]$. Product of L terms $\rightarrow 0$ exponentially. Early layers learn nothing.

Solutions:

- Use ReLU activation (gradient = 1 for $z > 0$)
- Residual / skip connections (ResNets)
- Careful weight initialisation (Xavier, He)
- Use LSTM gates instead of vanilla RNN

Exploding Gradient

Cause:

If weights > 1 , product of L terms $\rightarrow \infty$. Loss becomes NaN. Network diverges during training.

Solutions:

- Gradient clipping: if $||\nabla|| > \text{threshold}$, scale ∇ down
- Smaller learning rate
- Careful weight initialisation
- Batch Normalisation

Rule of thumb: if training loss is stuck, suspect vanishing gradient. If loss is NaN or exploding, suspect exploding gradient $\rightarrow$ clip or reduce lr.

SGD, Momentum, RMSProp, Adam — Compared

SGD

$$w \leftarrow w - \eta \nabla L$$

✓ Simple, interpretable. Generalises well with right lr schedule.

✗ Slow convergence, sensitive to learning rate η . Oscillates in ravines.

📌 Large-scale vision models with careful lr scheduling (cosine annealing)

Momentum

$$v \leftarrow \beta v - \eta \nabla L; w \leftarrow w + v$$

✓ Accelerates along consistent gradient directions. Dampens oscillations.

✗ Extra hyperparameter β (typically 0.9). Can overshoot minima.

📌 Good general improvement over vanilla SGD

RMSProp

$$s \leftarrow \rho s + (1-\rho) \nabla L^2; w \leftarrow w - \eta \nabla L / \sqrt{s + \epsilon}$$

✓ Adapts lr per-parameter. Good for RNNs and non-stationary objectives.

✗ Not as popular as Adam. ρ (decay rate) needs tuning.

📌 RNNs, LSTMs, non-stationary loss landscapes

Adam

$$m \leftarrow \beta_1 m + (1-\beta_1) \nabla L; v \leftarrow \beta_2 v + (1-\beta_2) \nabla L^2 \\ w \leftarrow w - \eta \cdot \hat{m} / \sqrt{\hat{v} + \epsilon}$$

✓ Fast convergence. Self-tunes lr per-parameter. Robust to hyperparams.

✗ Can overfit. May generalise slightly worse than SGD on some vision tasks.

📌 Default choice for most deep learning tasks

Batch Normalisation & Dropout — In Depth

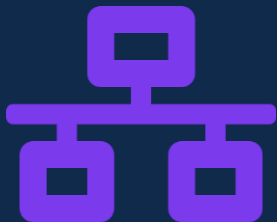
Batch Normalisation (Ioffe & Szegedy, 2015)

Algorithm (per mini-batch): $\mu_B = \text{mean}(x_B)$ $\sigma^2_B = \text{var}(x_B)$ $\hat{x}_i = (x_i - \mu_B) / \sqrt{\sigma^2_B + \epsilon}$ $y_i = \gamma \hat{x}_i + \beta$

- Normalises pre-activations → reduces internal covariate shift → faster training
- Acts as regulariser → often reduces need for Dropout
- Allows higher learning rates without divergence
- γ and β are learnable — network can undo normalisation if needed

Dropout (Srivastava et al., 2014)

- During training: each neuron is independently zeroed with probability p (dropout rate, typically 0.2–0.5)
- During inference: all neurons active, but outputs scaled by $(1-p)$ to maintain expected values
- Effectively trains an exponential ensemble of 2^n sub-networks simultaneously
- Common: $p=0.5$ for FC layers, $p=0.1-0.2$ for convolutional layers



LECTURE 2 · SECTION 2.5

RNNs, LSTMs & Transformer Models

Gate equations · Seq2seq · Attention mechanism · BERT · Practical NLP

LSTM — Full Mathematical Specification

Forget Gate

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Decides what fraction of the previous cell state C_{t-1} to retain. Output $\in (0,1)$ per cell.

Input Gate

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

Decides what new information to write to the cell state. i_t gates the candidate $\tilde{C}_t$.

Cell State Update

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

The cell state C_t flows through time with only element-wise multiplication and addition — enabling gradient flow over long sequences.

Output Gate

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(C_t)$$

Decides what to output as the hidden state h_t — the 'memory' passed to the next time step and used for prediction.

Key insight: The cell state C_t acts as a conveyor belt — information can flow unchanged across many time steps. The forget gate can keep gradients alive: if $f_t \approx 1$, then $\partial C_t / \partial C_{t-1} \approx 1$ — no vanishing!

Attention Mechanism & BERT

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$$

Q (Query)

What the current token is looking for from context

K (Key)

What each other token offers as searchable information

V (Value)

What each other token actually contributes if attended to

$\sqrt{d_k}$ scaling

Prevents dot products from growing too large → softmax saturation

BERT — Bidirectional Encoder Representations from Transformers (Google, 2018)

- Architecture: 12–24 stacked Transformer encoder blocks, 110M–340M parameters
- Pre-training: Masked Language Modelling (MLM) — predict 15% of masked tokens on 3.3B word corpus
- Fine-tuning: add a task-specific head (classification, NER, QA) and fine-tune entire model on labelled data (even with 1K examples)
- Contextual embeddings: same word 'bank' gets different vectors in 'river bank' vs 'bank account'
- State of the art: outperformed previous SOTA on 11 NLP tasks simultaneously when released

Modern equivalents: RoBERTa (improved training), DeBERTa (disentangled attention), GPT-4 (decoder-only, generative)



LECTURE 2 · SECTION 2.6

Advanced Clustering & Evaluation Methods

*GMM · EM algorithm · Spectral clustering · ROC/PR deep dive ·
Statistical tests*

GMM — Soft Probabilistic Clustering

$$p(x) = \sum_{k=1}^K \pi_k \cdot N(x \mid \mu_k, \Sigma_k) \quad \text{where } \pi_k = \text{mixing coefficient, } \sum_k \pi_k = 1$$

E-step (Expectation)

Compute the responsibility of cluster k for each point x_i :

$$r_{ik} = \pi_k N(x_i \mid \mu_k, \Sigma_k) / \sum_j \pi_j N(x_i \mid \mu_j, \Sigma_j)$$

→ Soft assignment: each point belongs to ALL clusters with different probabilities

M-step (Maximisation)

Update parameters using weighted statistics:

$$N_k = \sum_i r_{ik} \quad \mu_k = (1/N_k) \sum_i r_{ik} x_i \quad \Sigma_k = (1/N_k) \sum_i r_{ik} (x_i - \mu_k)(x_i - \mu_k)^T \quad \pi_k = N_k/N$$

→ Each cluster's parameters are weighted by how responsible it is for each point

Convergence

Repeat E→M until log-likelihood $\sum_i \log p(x_i)$ stops increasing.

EM is guaranteed to increase the likelihood at each step (but may find local optima).

Multiple random restarts recommended — keep solution with highest final likelihood

GMM vs k-Means: k-Means is a hard-assignment special case of GMM (spherical $\Sigma_k = \sigma^2 I$). GMM is more flexible (elliptical clusters), provides probability of cluster membership, and uses AIC/BIC to choose k .

Spectral Clustering — Graph-Based Approach

1 Build Similarity Graph

Compute affinity matrix W where $W_{ij} = \exp(-||x_i - x_j||^2 / 2\sigma^2)$ between all pairs (or use k-NN graph: connect only k nearest neighbours)

2 Compute Graph Laplacian

$L = D - W$ where D is degree matrix ($D_{ii} = \sum_j W_{ij}$)
Normalised: $L_n = D^{-1/2} L D^{-1/2}$
Encodes the connected structure of the data graph

3 Eigendecomposition

Compute the k smallest eigenvectors of L (or L_n)
Stack them as columns $\rightarrow$ form $n \times k$ matrix U
Each row of U is the 'spectral embedding' of a point

4 Cluster Embeddings

Apply k-Means to the rows of U
The spectral embedding linearises non-linear cluster structures
Result: can find crescent, ring, and interleaved clusters!

When to use: Spectral clustering excels on non-convex cluster shapes (rings, crescents, interleaved spirals) where k-Means and GMM fail. Cost: $O(n^3)$ eigendecomposition — does not scale to very large n . Use approximate methods (Nyström) for $n > 10,000$.

ROC vs Precision-Recall — When to Use Which

Aspect	ROC Curve	Precision-Recall Curve
X-axis	False Positive Rate ($FPR = FP/(FP+TN)$)	Recall ($= TPR = TP/(TP+FN)$)
Y-axis	True Positive Rate ($TPR = TP/(TP+FN)$)	Precision ($= TP/(TP+FP)$)
Baseline (random)	Diagonal from (0,0) to (1,1) — $AUC = 0.5$	Horizontal line at $y = P/(P+N)$ — prevalence
Best when	Balanced classes, FP and FN equally costly	Imbalanced classes, minority class is the focus
Summary metric	AUC-ROC (area under ROC curve)	AUC-PR or Average Precision (AP)
Interpretation	$P(\text{score}(\text{pos}) > \text{score}(\text{neg}))$	Harmonic mean of P & R across thresholds

Threshold Selection

Default: 0.5 (balanced P & R)
Youden's J: $\text{argmax}(TPR - FPR)$ (ROC-optimal)
F-beta optimal: argmax $F\beta$ score on val set
Cost-sensitive: choose threshold to min. business cost

Fraud Detection Example

1M transactions, 1,000 fraudulent (0.1%).
 $AUC-ROC = 0.99$ — looks great!
But $AUC-PR = 0.45$ — model misses 55% of fraud.
→ Always use AUC-PR for severe class imbalance!

Is My Model Actually Better? Statistical Tests

McNemar's Test

Use when: Compare two classifiers on the SAME test set

$\chi^2 = (|b-c| - 1)^2 / (b+c)$ where $b = A$ correct, B wrong; $c = B$ correct, A wrong

H_0 : both models make the same errors

$p < 0.05 \rightarrow$ statistically significant difference



Two models, same fixed test set, classification tasks

Paired t-Test / Wilcoxon

Use when: Compare across k-fold cross-validation scores

$t = \bar{d} / (sd / \sqrt{k})$ where d = score differences across folds

Wilcoxon signed-rank test if normality assumption is doubtful

Correction: use 5x2 CV test (Dietterich, 1998) for better type-I error



CV results, non-normal distributions $\rightarrow$ use Wilcoxon

Friedman + Nemenyi

Use when: Compare multiple (≥ 3) algorithms across datasets

Friedman: non-parametric ANOVA — tests if any algorithm differs

Nemenyi: post-hoc test for pairwise comparisons with Bonferroni correction

Standard in ML competition benchmarks and NeurIPS papers



Algorithm comparison papers, benchmark studies

⚠ Never report a single train/test split result as conclusive. Always use k-fold CV with statistical testing for published results.

How to Understand What Your Model Has Learned



Permutation Importance

Model-agnostic

Shuffle each feature column, measure drop in performance. The bigger the drop, the more important the feature.

✓ Works for any model · accounts for interactions

✗ Slow for many features · can be misleading with correlated features



SHAP Values

Model-agnostic (game theory)

Assigns each feature its average marginal contribution across all possible feature orderings (Shapley values from cooperative game theory).

✓ Theoretically grounded · consistent · local + global

✗ Slow for large datasets without tree approximations



RF Feature Importance

Embedded (tree models)

Mean decrease in Gini impurity (MDI) across all trees. Or: mean decrease in accuracy (MDA) — equivalent to permutation importance.

✓ Very fast · computed during training

✗ Biased towards high-cardinality features with MDI · use MDA for reliability

Recommended workflow: Train RF → MDI for fast screening → Permutation importance to verify → SHAP for individual prediction explanations.

Handling Class Imbalance — Complete Toolkit

Random Under-sampling

Resampling

Randomly remove majority class samples until balanced. Fast.
Risk: losing informative majority samples.

`RandomUnderSampler(sampling_strategy=0.5)`

SMOTE

Synthetic Over-sampling

Interpolate between minority class neighbours to create synthetic samples. Better than random duplication.

`SMOTE(k_neighbors=5, random_state=42)`

ADASYN

Adaptive Synthetic

Like SMOTE but generates more synthetic samples in harder-to-learn regions of the feature space.

`ADASYN(sampling_strategy='auto')`

Class Weights

Loss Re-weighting

Multiply the loss by w_{neg}/w_{pos} for minority class — no data augmentation needed. Elegant and fast.

`SVC(class_weight='balanced')` or `XGBoost scale_pos_weight`

Ensemble Rebalancing

Algorithmic

BalancedBaggingClassifier: each bootstrap bag is balanced.
EasyEnsemble: uses many balanced sub-ensembles.

`BalancedBaggingClassifier(base_estimator=DT)`

Threshold Tuning

Post-processing

Move decision boundary from 0.5 to favour minority recall.
Use Precision-Recall curve to find optimal F1 threshold.

`threshold = optimal_f1_threshold(y_val, scores)`

Complete scikit-learn Pipeline — Best Practices

```
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.metrics import classification_report

# Step 1: Define preprocessing per column type
num_pipe = Pipeline([['impute', SimpleImputer(strategy='median')),
                      ('scale', StandardScaler())])
cat_pipe = Pipeline([['impute', SimpleImputer(strategy='most_frequent')),
                      ('encode', OneHotEncoder(handle_unknown='ignore'))])
preprocessor = ColumnTransformer([
    ('num', num_pipe, num_cols), ('cat', cat_pipe, cat_cols)])

# Step 2: Full pipeline with model
pipe = Pipeline([['prep', preprocessor], ('clf', RandomForestClassifier(n_estimators=200))])

# Step 3: Evaluate with stratified CV
cv_results = cross_validate(pipe, X, y, cv=StratifiedKFold(5),
                             scoring=['f1_weighted', 'roc_auc'], return_train_score=True)
print(f"F1: {cv_results['test_f1_weighted'].mean():.3f} ± {cv_results['test_f1_weighted'].std():.3f}")
```

Making Your Neural Network Train Well

Weight Initialisation

- Xavier/Glorot: scale weights by $\sqrt{2/(\text{fan_in} + \text{fan_out})}$ — good for Sigmoid/Tanh
- He initialisation: scale by $\sqrt{2/\text{fan_in}}$ — correct for ReLU
- Wrong init → vanishing or exploding gradients from the start

Learning Rate Schedule

- Cosine annealing: lr decays as $\text{lr_min} + 0.5(\text{lr_max} - \text{lr_min})(1 + \cos(\pi t/T))$
- Warmup: linearly increase lr for first k steps to avoid early instability
- OneCycleLR (fast.ai): combines warmup + cosine decay in one cycle

Gradient Clipping

- Clip gradient NORM: if $\|\nabla\| > \text{max_norm}$, scale ∇ by $\text{max_norm}/\|\nabla\|$
- Clip gradient VALUE: clamp each element to $[-\text{clip_val}, \text{clip_val}]$
- Standard for RNNs/LSTMs. Also helps CNNs with unusual loss landscapes

Data Augmentation

- Images: random crop, flip, colour jitter, mixup, CutMix, RandAugment
- Text: back-translation, synonym replacement, EDA, GPT-based paraphrase
- Tabular: SMOTE, Gaussian noise injection, feature dropout during training

Learning Rate Finder

- Exponentially increase lr over one pass, record loss. Use lr just before loss explosion.
- Smith (2017) learning rate range test — implemented in fastai and PyTorch Lightning
- Typical good lr: 10× smaller than the lr at minimum loss

Early Stopping

- Monitor `val_loss`; stop if no improvement for 'patience' epochs
- Save the best checkpoint (ModelCheckpoint callback)
- Typical patience: 10–20 epochs. Restore best weights at end

Beyond Lift — Advanced Rule Quality Measures

Lift

Range: $[0, \infty)$

$$\text{Conf}(X \rightarrow Y) / \text{Supp}(Y)$$

= 1: independent. > 1: positive correlation. < 1: negative correlation.

⚠ Not symmetric: $\text{Lift}(X \rightarrow Y) = \text{Lift}(Y \rightarrow X)$

Conviction

Range: $[0, \infty)$

$$(1 - \text{Supp}(Y)) / (1 - \text{Conf}(X \rightarrow Y))$$

> 1: X implies Y strongly. = 1: X and Y independent. $\rightarrow \infty$: perfect rule.

⚠ Directional (asymmetric). Better than Lift for strong rules.

Leverage

Range: $[-0.25, 0.25]$

$$\text{Supp}(XUY) - \text{Supp}(X) \cdot \text{Supp}(Y)$$

> 0: items co-occur more than expected by chance under independence.

⚠ Biased towards frequent items. Use with Lift for filtering.

Kulczynski (Kulc)

Range: $[0, 1]$

$$[\text{Conf}(X \rightarrow Y) + \text{Conf}(Y \rightarrow X)] / 2$$

Symmetric. High Kulc = strong bidirectional relationship.

⚠ Pair with Imbalance Ratio to detect skewed associations.

Cosine Similarity

Range: $[0, 1]$

$$\text{Supp}(XUY) / \sqrt{\text{Supp}(X) \cdot \text{Supp}(Y)}$$

= 1: perfect co-occurrence. Symmetric. Geometric mean of confidences.

⚠ Resistant to high-support dominance bias.

Imbalance Ratio (IR)

Range: $[0, 1]$

$$|\text{Supp}(X) - \text{Supp}(Y)| / (\text{Supp}(X) + \text{Supp}(Y) - \text{Supp}(XUY))$$

= 0: balanced supports. = 1: one item always implies the other.

⚠ Used with Kulc to detect unbalanced directional rules.

Lecture 2 — Key Concepts Checklist

2.1 Naive Bayes

Laplace smoothing formula and rationale · log-probability trick · Spam worked example · Gaussian NB for continuous features

2.2 Decision Trees

Information Gain worked example (Outlook = 0.247 bits) · CART vs ID3 vs C4.5 comparison · Gain Ratio formula

2.3 SVM

Lagrangian dual formulation · KKT conditions · Support vectors definition · C and γ effects · OvR vs OvO multi-class

2.4 Neural Network Training

Vanishing/exploding gradients — causes and fixes · SGD/Momentum/RMSProp/Adam comparison · Batch Norm algorithm · Dropout mechanics

2.5 RNNs & Transformers

LSTM gate equations (forget, input, cell, output) · Attention: Q, K, V formulation · BERT architecture and pre-training tasks

2.6 Advanced Methods

GMM + EM algorithm · Spectral clustering 4-step process · ROC vs PR curve guidance · McNemar/Wilcoxon/Friedman tests · SHAP/permutation importance

Lecture 2 — Practice Examination Questions

Q1. Using the Naive Bayes classifier, classify the email 'win free money' as Spam or Ham given the training corpus from Slide 4. Show all calculation steps including Laplace smoothing.

12 pts

Q2. Compute the Information Gain for splitting on feature 'Wind' (Weak/Strong) for the Tennis dataset where Weak→6Yes/2No and Strong→3Yes/3No. Is Outlook still the better split?

10 pts

Q3. Explain why vanishing gradients occur in deep networks using Sigmoid activations. Describe THREE distinct solutions and explain the mechanism by which each resolves the problem.

12 pts

Q4. Derive the LSTM forget gate equation and explain what happens to the cell state when: (a) $f_t \approx 0$, (b) $f_t \approx 1$. Why does this help with the vanishing gradient problem?

10 pts

Q5. A model achieves AUC-ROC = 0.99 on a dataset where 0.1% of samples are positive. Does this guarantee the model is good at detecting positives? Propose TWO better evaluation approaches.

8 pts

Q6. Compare GMM with k-Means: (a) What additional flexibility does GMM provide? (b) Write the EM E-step equation. (c) When would you prefer k-Means?

13 pts

Upcoming — Lecture 3 Topics



Advanced Deep Learning

GAN architecture, VAEs, self-supervised learning (SimCLR, MoCo), diffusion models overview.



Advanced NLP & RAG

Sentence transformers, vector databases, Retrieval-Augmented Generation, LLM fine-tuning.



Graph Neural Networks

Message passing, GCN, GAT, GraphSAGE. Applications: drug discovery, social network analysis.



Federated & Privacy-Aware ML

Federated learning algorithms, differential privacy, secure multi-party computation.



ML in Production (MLOps)

Model versioning, drift detection, A/B testing, CI/CD for ML, model serving at scale.



Coursework Project Kickoff

Dataset selection, baseline establishment, project milestones, code repository setup.

Before Lecture 3: Complete Lab Assignment 1 (k-NN + Naive Bayes on provided dataset). Submit via LMS by the due date.



End of Lecture 2

CSD622 · Pattern Recognition & Data Mining

*Naive Bayes · Decision Trees · SVM · Neural Network Training · LSTMs ·
GMMs · Evaluation*

34

New Slides

8

Worked Examples

3

Code Snippets

6

Exam Questions

Tutor: Dr A.J Fofanah